Manual de SQL (Structured Query Language) para el análisis de datos (Con ejemplo práctico) – Caso práctico 1

Autor

Juan Pablo Gutiérrez Bernal

Calificador:

María Eugenia Vásquez

Universidad Cesuma

Maestría En Ciencia de Datos Para Negocios (Big Data & Business Analytics)

México - Colombia

2026

Tabla de Contenido

Introducción 3

1. Descripción de la herramienta 3

1.1 Definición 3

1.2 Historia breve 4

1.3 Motores populares 5

2. Funciones más destacadas 6

2.1 DDL (Data Definition Language) 6

2.2 DML (Data Manipulation Language) 8

2.3 DQL (Data Query Language) 9

2.4 DCL (Data Control Language) 11

2.5 Funciones de agregación y análisis estadístico 11

2.6 Programación procedural y lógica del servidor 12

3. Aplicaciones 13

3.1 Data wrangling (Procesamiento del raw data) 13

3.2 Business intelligence 14

3.3 Automatización 15

4. Ejemplo práctico 16

4.1 Creación de las variables y tablas 16

4.2 Ingreso de información 21

4.2.1 Ingreso de la información por clausula “INSERT INTO” 21

4.2.1 Ingreso de la información por clausula “INSERT INTO” 24

4.3 Consultas 27

Introducción

En el vasto ecosistema de la Ciencia de Datos, SQL (Structured Query Language) se posiciona como el lenguaje universal para la comunicación con bases de datos relacionales. A diferencia de lenguajes como Python o R, enfocados en el procesamiento y modelado, SQL es la herramienta esencial para la recuperación, manipulación y organización de grandes volúmenes de datos estructurados. Este manual explora sus fundamentos, desde el diseño de las bases de datos, consultas y mostrando un ejemplo práctico y completo, demostrando por qué sigue siendo la habilidad más demandada en el perfil de un Científico de Datos.

1. **Descripción de la herramienta**

1.1 Definición

El S.Q.L. (Structured Query Language) es el lenguaje estándar por excelencia para el diseño, consulta y manipulación de información en bases de datos relacionales empresariales. A diferencia de los lenguajes de programación convencionales, SQL destaca por su sintaxis intuitiva y declarativa, optimizada específicamente para interactuar con bases de datos.

Este lenguaje cuenta con un repertorio aproximado de 60 comandos esenciales que permiten al usuario ejecutar un amplio espectro de tareas críticas:

Arquitectura y diseño: Permite estructurar bases de datos desde su origen, definiendo la integridad de la información mediante el establecimiento de llaves primarias (primary keys) y foráneas (foreign keys).

Gestión de relaciones: Facilita la auditoría y consulta del modelo de datos para comprender las conexiones lógicas entre las diferentes tablas que conforman el sistema.

Interoperabilidad de datos: Ofrece flexibilidad para la ingesta de información, ya sea mediante inserciones manuales o la integración de archivos externos (Excel, CSV) y scripts de automatización en lenguajes como Python.

Automatización y limpieza: Permite la actualización dinámica de registros (comando UPDATE) y la implementación de triggers para automatizar procesos. Asimismo, es una herramienta clave para el análisis exploratorio (EDA) y la normalización de datos (Data Cleaning), permitiendo alterar tipos de datos y estructuras de columnas según se requiera.

Análisis avanzado: Posee una robusta gama de comandos para el cruce de información compleja mediante el uso de Joins, subconsultas y funciones de ventana (Window Functions).

Cálculo estadístico integrado: Incluye funciones de agregación fundamentales como SUM(), MAX(), MIN() y COUNT(), permitiendo obtener métricas clave de forma inmediata desde el servidor.

1.2 Historia breve

Para entender por qué las empresas usan SQL hoy, hay que imaginar cómo era el mundo en los años 70. En ese entonces, trabajar con datos en una computadora era una pesadilla; no había orden y cada quien guardaba la información como podía, lo que hacía casi imposible encontrar algo rápido.

Todo dio un giro en 1970 gracias a Edgar F. Codd, un investigador de IBM que propuso el Mode lo Relacional. Su propuesta era sencilla: organizar la información en tablas conectadas entre sí a través de claves comunes, como si fueran piezas de un rompecabezas que encajan perfectamente.

Basándose en esa idea, IBM creó un lenguaje llamado SEQUEL, que con los años terminó siendo el SQL que se conoce hoy en día. Lo que nació como un experimento de laboratorio se convirtió en el "idioma universal" de los datos. Es increíble pensar que, más de 50 años después, sigue siendo la herramienta que permite que cualquier empresa, por grande que sea, pueda hablar con sus datos de forma civilizada.

1.3 Motores populares

Ahora bien, se establece que S.Q.L es el lenguaje general, es el corazón de todas las bases de datos contemporáneas, sin embargo, hay múltiples formas de implementar este lenguaje. A través de los últimos años, diferentes empresas han creado sus propias aplicaciones para crear y manipular sus bases de datos, aplicaciones que cuentan con sus propias ventajas y funcionalidades, entre las más famosas se encuentran:

PostgreSQL: Considerado el motor de código abierto más avanzado del mercado. Destaca por su robustez y versatilidad en la ciencia de datos, ya que permite el procesamiento eficiente de tipos de datos no convencionales, como estructuras JSON y objetos geográficos.

MySQL: Es el sistema más extendido para el desarrollo de aplicaciones web. Su arquitectura está optimizada para ofrecer alta velocidad de respuesta y facilidad de configuración.

Microsoft SQL Server: Representa la opción predilecta en entornos corporativos que operan bajo el ecosistema de Microsoft. Su principal fortaleza radica en la integración nativa con herramientas de inteligencia de negocios como Power BI, facilitando la creación de flujos de trabajo analíticos y soporte técnico especializado.

Oracle Database: Se posiciona como la solución líder para el manejo de volúmenes masivos de datos en sectores críticos, tales como la banca y las telecomunicaciones. Al ser una plataforma licenciada, ofrece niveles superiores de escalabilidad y seguridad industrial para organizaciones que requieren una disponibilidad del servicio cercana al 100%.

SQLite: A diferencia de los motores cliente-servidor, SQLite es una biblioteca de software que representa una base de datos completa en un solo archivo de disco. Su diseño ligero lo convierte en el estándar para el almacenamiento local en aplicaciones móviles, navegadores web y dispositivos inteligentes.

2. Funciones más destacadas

Como ya se mencionó anteriormente, S.Q.L cuenta con múltiples comandos, cuyas finalidades dependen de lo que el usuario desee realizar con sus datos, es decir, se podría decir que los comandos se pueden dividir en varias categorías como DDL (Data Definition Language), DML (Data Manipulation Language, DQL (Data Query Language), DCL (Data Control Language), Agreggation Functions, Programación Procedural y Lógica del Servidor. A continuación, se verán los comandos más usados por cada categoría, una explicación sencilla seguida de un ejemplo en la aplicación de código abierto MySQL.

2.1 DDL (Data Definition Language)

Estos comandos son sencillos, se usan básicamente para la configuración inicial y el mantenimiento de la arquitectura de las bases de datos, algunos de los comandos más usados para este apartado son:

- **CREATE TABLE:** Define una nueva tabla especificando nombres y columnas y tipos de datos (entero, texto, fecha, decimal). En este paso se definen las primary keys y la foreign keys (recalcando que primero se hacen las tablas que no contengan foreign keys ya que puede resultar en un error si se crea una tabla que contenga una foreign key de una tabla que no se ha creado)

```
9   -- Tabla de empleados
10  CREATE TABLE empleados (
11    EMP_No INT PRIMARY KEY,
12    Apellido VARCHAR(100),
13    Oficio VARCHAR(100),
14    Director INT,
15    Fecha DATETIME NOT NULL,
16    Salario FLOAT NOT NULL,
17    Comision FLOAT,
18    Dep_no INT NOT NULL,
19    FOREIGN KEY (Dep_no) REFERENCES Departamentos(Dep_no)
20  );
```

Comando para crear la tabla

El cuerpo de la tabla, aquí se definen los tipos de datos, el numero de columnas y la primary key

Al final definimos las foreign keys, pueden ser 1 o mas de ser requerido

- **ALTER TABLE:** Permite agregar nuevas columnas, eliminar las existentes o modificar restricciones sin necesidad de borrar la tabla.

```
ALTER TABLE empleados
ADD Email VARCHAR(150);
ALTER TABLE empleados
MODIFY Oficio VARCHAR(255);
ALTER TABLE empleados
DROP COLUMN Comision;
ALTER TABLE empleados
ADD CONSTRAINT chk_salario CHECK (Salario > 0);
ALTER TABLE empleados
RENAME COLUMN Director TO Jefe_ID;
```

Estos son algunos ejemplos de lo que se puede hacer con ALTER TABLE, cabe aclarar que de ser necesario se puede escribir una sola QUERY, sin necesidad de separarlos por “;”, sino usando “,”. Es un comando sumamente importante al realizar el Data Cleaning.

- **CREATE INDEX:** Utilizado para mejorar la velocidad de búsqueda y recuperación de datos en las columnas que se consultan con frecuencia (Se puede crear un solo índice para varias columnas). El uso de este comando es recomendado para grandes volúmenes de datos, específicamente en columnas que tiendan a estar incluidas en clausulas “JOIN” o “WHERE”, ya que estas pueden congelar el sistema de bases de datos. Por último, no se recomienda crear índices para todas las columnas de las bases de datos ya que entorpece las operaciones de escritura como “INSERT” o “UPDATE”.

```
CREATE INDEX idx_dep_oficio
ON empleados (Dep_no, Oficio);
```

- **TRUNCATE:** Elimina todos los registros de una tabla de forma rápida, pero mantiene la estructura intacta para futuros usos.

```
TRUNCATE TABLE empleados;
```

2.2 DML (Data Manipulation Language)

Son las instrucciones destinadas a la interacción directa con la información almacenada, algunos de los comandos más usados para este apartado son:

- **INSERT INTO:** Es la función por excelencia para llenar una tabla vacía o incluir nuevos registros en las tablas existentes.

```
INSERT INTO empleados(EMP_no, Apellido, Oficio, Director, Fecha, Salario, Comision, Dep_no)
VALUES
(7499, 'ALONSO', 'VENDEDOR', 7698, STR_TO_DATE('20/02/81', '%d/%m/%y'), 140000, 40000, 30),
(7521, 'LOPEZ', 'EMPLEADO', 7782, STR_TO_DATE('08/05/81', '%d/%m/%y'), 135000, 0, 10),
(7654, 'MARTIN', 'VENDEDOR', 7698, STR_TO_DATE('28/09/81', '%d/%m/%y'), 150000, 160000, 30),
(7698, 'GARRIDO', 'DIRECTOR', 7839, STR_TO_DATE('01/05/81', '%d/%m/%y'), 385000, 0, 30),
(7782, 'MARTINEZ', 'DIRECTOR', 7839, STR_TO_DATE('09/06/81', '%d/%m/%y'), 245000, 0, 10),
(7839, 'REY', 'PRESIDENTE', null ,STR_TO_DATE('17/11/81', '%d/%m/%y'), 600000, 0, 10),
(7844, 'CALVO', 'VENDEDOR', 7698, STR_TO_DATE('08/09/81', '%d/%m/%y'), 180000, 0, 30),
(7876, 'GIL', 'ANALISTA', 7782, STR_TO_DATE('06/05/82', '%d/%m/%y'), 335000, 0, 20),
(7900, 'JIMENEZ', 'EMPLEADO', 7782, STR_TO_DATE('24/03/83', '%d/%m/%y'), 140000, 0, 20)
;
```

Aquí va el INSERT INTO, se debe llamar a la tabla, seguido de todas las columnas entre "()" a las que se quiera agregar informacion separadas por ",".

El comando VALUE le indica a SQL que se van a ingresar los valores.

Este es el cuerpo del comando, aquí se separan por filas, y dentro de las listas de elementos se ingresan de acuerdo al orden de las columnas que establecimos anteriormente ya que si se ingresa un dato de tipo STRING a una columna de tipo INT, resultará en error.

- **UPDATE:** Modifica valores específicos dentro de las tablas. Se utiliza comúnmente con la cláusula WHERE para evitar la alteración accidental de toda la base de datos.

```
UPDATE empleados
SET Salario = Salario * 1.1
WHERE Dep_no = 30;
```

- **DELETE:** Elimina registros específicos dentro de las tablas, se debe tener cuidado al igual que en UPDATE con los filtros para no alterar todos los valores y se usa comúnmente con WHERE (No confundir con DROP, ya que DROP es para objetos y WHERE para datos específicos).

```
DELETE FROM empleados
WHERE EMP_No = 7369;
```

2.3 DQL (Data Query Language)

Es el área de mayor relevancia durante el análisis de datos, ya que permite transformar datos crudos en información útil para las empresas. Algunos de los comandos más usados para este apartado son:

- **SELECT (DISTINCT):** SELECT es el comando por excelencia al hacer una consulta en una base de datos, es el que inicia la consulta. Normalmente va seguido de los nombres de las columnas que quieres mostrar en la consulta o un * si quieres mostrar todas las columnas, va acompañado del FROM, que es el comando que especifica de cual tabla quieres extraer esas columnas (El DISTINCT se usa cuando queremos eliminar valores duplicados de la consulta).

```
8  SELECT *
9  FROM empleados;
```

- **WHERE:** Es una clausula de filtrado, teniendo como complementos operadores lógicos como AND, OR, IN, BETWEEN, lo que permite crear filtros para segmentar información compleja.

```
8  SELECT *
9  FROM empleados
10 WHERE Salario > 100000 AND Salario < 150000;
```

- **ORDER BY:** Organiza los resultados de forma ascendente o descendente según una o más columnas. Por defecto lo hace de forma ascendente.

```
SELECT *
FROM empleados
WHERE Salario > 100000 AND Salario < 150000
ORDER BY Apellido desc;
```

- **JOIN:** Permite combinar filas de dos o más tablas basándose en una columna común entre ellas. Existen varios tipos de JOINS y se usan dependiendo de lo que necesite el usuario, algunos de estos son INNER JOIN, OUTER JOIN, FULL OUTER JOIN, LEFT JOIN, RIGHT JOIN etc. La estructura varia en pequeños puntos dependiendo del JOIN que queramos, pero lo principal es que las dos tablas deben compartir una columna, y se acompaña con el comando ON para señalarle a SQL donde hacer el JOIN. Es una de las funciones más importantes para los analistas de datos.

```
SELECT empleados.Apellido, Departamentos.Nombre_Dep
FROM empleados
INNER JOIN Departamentos
ON empleados.Dep_no = Departamentos.Dep_no;
```

- **GROUP BY:** Su propósito principal es permitir la aplicación de agreggation functions (Funciones de agregación) como MAX, COUNT, AVG, SUM, MIN sobre los datos, transformando conjuntos detallados de registros en resúmenes estadísticos.

```
SELECT Dep_no, AVG(Salario) AS Salario_Promedio
FROM empleados
GROUP BY Dep_no;
```

- **LIMIT/TOP:** Restringe el número de filas devueltas, algo vital cuando se trabaja con tablas de millones de filas y registros en BIG DATA.

```
SELECT Apellido, Salario, Oficio
FROM empleados
ORDER BY Salario DESC
LIMIT 5;
```

2.4 DCL (Data Control Language)

Son funciones exclusivamente hechas para los permisos de los usuarios en las bases de datos, es decir, es netamente administrativo:

- **GRANT:** Otorga permisos específicos a usuarios para que puedan realizar consultas o modificaciones.
- **REVOKE:** Elimina los permisos previamente concedidos, asegurando que solo el personal autorizado acceda a datos sensibles.

2.5 Funciones de agregación y análisis estadístico

Dentro de la extracción de datos, SQL ofrece herramientas para el resumen masivo de información, estas funciones procesan conjuntos de valores para devolver métricas claves.

- **SUM:** Es la función de SQL que nos permite sumar para calcular totales.
- **AVG:** Calcula el promedio de los datos
- **COUNT:** Como su nombre lo indica, nos facilita contar el número de registros o filas.
- **MAX:** Función que nos permite calcular el registro más grande de un conjunto de valores
- **MIN:** Función que nos permite calcular el registro más pequeño de un conjunto de valores

```sql
SELECT
    Oficio,
    COUNT(*) AS Cantidad_Empleados,
    SUM(Salario) AS Costo_Total_Salarial,
    AVG(Salario) AS Salario_Promedio,
    MIN(Salario) AS Salario_Minimo,
    MAX(Salario) AS Salario_Maximo
FROM empleados
GROUP BY Oficio;
```

2.6 Programación procedural y lógica del servidor

Es el nivel más avanzado del lenguaje SQL, permitiendo integrar la lógica de la programación clásica directamente en el motor de la base de datos para automatizar tareas. Algunos de los más usados son:

- **Funciones y procedimientos almacenados:** Bloques de código reutilizable diseñados para ejecutar cálculos complejos o llevar a cabo funciones muy especificas dentro de una base de datos. Empieza con el comando CREATE PROCEDURE y va de la mano con comandos como BEGIN y END, además se debe hacer uso de un DELIMITER, esto aplica para las funciones, procedimientos y triggers. Para llamar al procedimiento o función debemos usar la cláusula CALL seguido del nombre del procedimiento.

```sql
DELIMITER //

CREATE PROCEDURE AumentarSalarioPorDepartamento (
    IN dep_id INT,
    IN porcentaje_aumento FLOAT
)
BEGIN
    UPDATE empleados
    SET Salario = Salario * (1 + (porcentaje_aumento / 100))
    WHERE Dep_no = dep_id;
END //

DELIMITER ;
```

- **Triggers (Disparadores):** Son mecanismos que, como su mismo nombre lo indica, se disparan ante un evento especifico dentro de la base de datos, ideales para automatizar tareas. Suele ir muy de la mano con las funciones y procedimientos almacenados.

```
DELIMITER //

CREATE TRIGGER audit_salario_update
BEFORE UPDATE ON empleados
FOR EACH ROW
BEGIN
    -- Se inserta un registro en la tabla de auditoría con los valores antes del cambio
    INSERT INTO Log_Salarial (EMP_No, Salario_Anterior, Fecha_Cambio)
    VALUES (OLD.EMP_No, OLD.Salario, NOW());
END //

DELIMITER ;
```

3. Aplicaciones

Dominar la sintaxis de SQL es el paso inicial, el valor real reside en su capacidad para resolver desafíos operativos y analíticos. En el entorno empresarial, SQL funciona como el motor de la arquitectura de datos, actuando como el vínculo directo entre los repositorios de Raw Data y el Business Intelligence.

3.1 Data wrangling (Procesamiento del raw data)

En el ámbito profesional, los datos en bruto suelen presentar inconsistencias, duplicidades, vacíos o errores de captura que invalidan cualquier análisis posterior. SQL se posiciona como la herramienta estándar para el data cleaning, permitiendo transformar el raw data en activos de información con alta integridad. Esto se divide en procesos principales:

- **Normalización y estandarización de atributos:** A través de la instrucción UPDATE, se corrigen las disparidades en los registros. Esto incluye la homologación de formatos, por ejemplo, convertir estados de "Pendiente", "pend" o "P" a un único valor estándar, y la corrección de errores tipográficos masivos. Este paso es vital para que las funciones

de agrupamiento (GROUP BY) no generen categorías duplicadas por errores de escritura.

- **Cleaning:** Mediante el uso de DELETE y filtros avanzados con WHERE, se eliminan registros que no cumplen con los estándares de calidad, como datos fuera de rango, registros de prueba o filas con valores nulos críticos. Este proceso garantiza que la "muestra" de datos sea representativa y no sesgue los resultados estadísticos de los reportes finales.
- **Consolidación y enriquecimiento de datos:** Rara vez la información necesaria reside en una sola tabla. El comando JOIN permite estructurar fuentes fragmentadas para crear conjuntos de datos integrales. Esta capacidad de "relacionar" tablas es lo que permite realizar análisis multidimensionales, obteniendo un punto de vista global de la situación general de la empresa.
- **Validación de consistencia mediante agregación:** Antes de dar por finalizada la curación, se utilizan funciones de agregación para verificar que, tras la limpieza, los volúmenes de datos y los rangos de valores sean coherentes con la realidad operativa, asegurando que no se haya perdido información crítica durante el proceso.

3.2 Business intelligence

Las plataformas de visualización no operan de forma aislada; su eficiencia depende directamente de consultas SQL optimizadas. En lugar de cargar millones de registros crudos en estas herramientas, SQL actúa como el motor de pre-procesamiento que sintetiza la información en la fuente. Esto se realiza mediante las siguientes características:

- **KPIs:** Mediante la combinación de GROUP BY y funciones de agregación, SQL transforma transacciones individuales en métricas de rendimiento. Esto permite calcular instantáneamente KPIs básicos, pero críticos inherentes de cada negocio o tipo

de empresa. La capacidad de procesar estos cálculos directamente en el servidor asegura que el Dashboard muestre información precisa y actualizada.

- **Análisis de tendencias y segmentación:** SQL permite segmentar la información por dimensiones temporales o categóricas. Esto convierte una base de datos estática en una herramienta de diagnóstico capaz de señalar exactamente dónde se están cumpliendo los objetivos y dónde existen cuellos de botella.
- **Optimización de reportes**: Un reporte profesional debe ser fluido. El uso de Views en SQL permite guardar consultas complejas que ya contienen las uniones (JOIN) y cálculos necesarios, útil para ahorrar tiempo. De este modo, la herramienta de BI solo necesita invocar la vista, reduciendo drásticamente los tiempos de carga.
- **Creación de capas de abstracción de datos:** SQL actúa como un filtro que selecciona únicamente datos necesarios para el negocio. Al entregar un conjunto de datos "limpio y resumido" a la capa de visualización, se facilita la creación de narrativas visuales que son fáciles de interpretar para directivos y socios estratégicos.

3.3 Automatización

En empresas de alto nivel, la gestión manual de los datos es muy propensa al error humano. SQL ofrece la posibilidad de programación interna que transforman una base de datos estática en un sistema dinámico y autónomo, capaz de procesar volúmenes masivos de información sin degradar su rendimiento ni comprometer la seguridad. Este pilar de escalabilidad se fundamenta en los siguientes mecanismos de control:

- **Arquitectura de alto rendimiento mediante índices:** A medida que una tabla crece de miles a millones de registros, la velocidad de respuesta se convierte en un factor crítico. El uso estratégico de índices funciona como el índice de un libro técnico, permitiendo que el motor localice la información sin necesidad de escanear toda la base de datos.

- **Encapsulamiento de Lógica de Negocio:** Los Procedimientos Almacenados permiten centralizar procesos complejos en el servidor. Al ejecutar operaciones desde el motor y no desde una aplicación externa, se reduce drásticamente el tráfico de red y esto facilita el mantenimiento, ya que cualquier cambio en la lógica se realiza en un solo lugar.
- **Garantía de Integridad y Auditoría:** Los Triggers actúan como centinelas automáticos que reaccionan ante eventos específicos. Son esenciales para mantener la trazabilidad, ya que pueden registrar automáticamente quién, cuándo y qué se modificó en una log table.
- **Gestión de Cargas de Trabajo Masivas:** SQL permite la automatización de tareas de mantenimiento, como la limpieza de registros temporales o la actualización de estadísticas de consulta durante horas de bajo tráfico. Esta capacidad de gestión autónoma asegura que la base de datos mantenga su salud operativa 24/7.

4. Ejemplo práctico

Durante el ejemplo práctico se usará una pequeña base de datos diseñada para una veterinaria ficticia. En este ejercicio, se podrá evidenciar el uso de la mayoría de funciones explicadas durante el transcurso de este manual, empezando por el diseño y la creación de la base de datos, lo más normalizada posible y con todas sus tablas bien relacionadas por el correcto uso de las llaves primarias y secundarias, pasando por las clausulas para llenar las tablas y un pequeño código de Python usado para generar x cantidad de registros en tablas con mayor requerimiento de información; y finalizando con la presentación de algunas clausulas para la consulta de esta base de datos.

4.1 Creación de las variables y tablas

En primera instancia se crearon todas las tables con clausulas “CREATE”, relacionando cada con sus llaves primarias o secundarias, es muy importante relacionar estas llaves ya que

servirán más adelante para las consultas. En total, se crearon 16 tablas para cubrir las necesidades del negocio, cabe aclarar que esto varia para cada negocio y sus necesidades, sumándole la capacidad del diseñador de la base de datos para normalizar lo mejor posible sus tablas, ya que así se evitará problemas a futuro cuando se deseen actualizar los datos dentro de las diferentes tablas.

Las tablas son las siguientes:

- Tabla de propietarios

```sql
-- 1. Tabla de propietarios
CREATE TABLE propietarios (
  id_propietario INT PRIMARY KEY AUTO_INCREMENT,
  propietarios_nombre VARCHAR(100) NOT NULL,
  propietarios_apellido VARCHAR(100) NOT NULL,
  direccion VARCHAR(150),
  propietarios_telefono VARCHAR(20),
  propietarios_email VARCHAR(100),
  fecha_registro DATE DEFAULT (CURRENT_DATE)
);
```

- Tabla de mascotas

```sql
-- 2. Tabla de mascotas
CREATE TABLE mascotas (
  id_mascota INT PRIMARY KEY AUTO_INCREMENT,
  nombre VARCHAR(50) NOT NULL,
  especie VARCHAR(50),
  raza VARCHAR(50),
  sexo CHAR(1),
  edad_estimada INT NOT NULL,
  peso_kg FLOAT,
  id_propietario INT,
  FOREIGN KEY (id_propietario) REFERENCES propietarios(id_propietario)
);
```

- Tabla de especialidad

```sql
-- 3. Tabla de especialidad
CREATE TABLE especialidad_vet (
 id_especialidad_vet INT PRIMARY KEY AUTO_INCREMENT,
 nombre_especialidad VARCHAR(100) NOT NULL
);
```

- Tabla de veterinarios

```sql
-- 4. Tabla de veterinarios
CREATE TABLE veterinarios (
 id_veterinario INT PRIMARY KEY AUTO_INCREMENT,
 veterinarios_nombre VARCHAR(100) NOT NULL,
 veterinarios_apellido VARCHAR(100),
 id_especialidad_vet INT NOT NULL,
 veterinarios_telefono VARCHAR(20),
 veterinarios_email VARCHAR(100),
 FOREIGN KEY (id_especialidad_vet) REFERENCES especialidad_vet(id_especialidad_vet)
);
```

- Tabla de estado de cita

```sql
,,
-- 5. Tabla de estado de cita
CREATE TABLE estado_cita (
    id_estado_cita INT PRIMARY KEY AUTO_INCREMENT,
    estado_de_cita VARCHAR(100)
);
```

- Tabla de motivo de la cita

```sql
-- 6. Tabla de motivo de la cita
CREATE TABLE motivo_cita (
    id_motivo_cita INT PRIMARY KEY AUTO_INCREMENT,
    motivo_de_cita VARCHAR(100)
);
```

- Tablas de citas

```sql
-- 7. Tabla de citas
CREATE TABLE citas (
 id_cita INT PRIMARY KEY AUTO_INCREMENT,
 id_mascota INT,
 id_veterinario INT,
 fecha_hora DATETIME NOT NULL,
 id_motivo_cita INT NOT NULL,
 descripcion VARCHAR(100),
 id_estado_cita INT NOT NULL,
 precio FLOAT NOT NULL,
 FOREIGN KEY (id_mascota) REFERENCES mascotas(id_mascota),
 FOREIGN KEY (id_veterinario) REFERENCES veterinarios(id_veterinario),
 FOREIGN KEY (id_estado_cita) REFERENCES estado_cita(id_estado_cita),
 FOREIGN KEY (id_motivo_cita) REFERENCES motivo_cita(id_motivo_cita)
);
```

- Tabla de servicios

```sql
-- 8. Tabla de servicios
CREATE TABLE servicios (
 id_servicio INT PRIMARY KEY AUTO_INCREMENT,
 nombre VARCHAR(100) NOT NULL,
 descripcion TEXT,
 precio DECIMAL(10,2)
);
```

- Tabla de estado de agendamiento

```sql
-- 9. TABLA DE ESTADO DE AGENDAMIENTO
CREATE TABLE estado_agendamiento (
    id_estado_agendamiento INT PRIMARY KEY AUTO_INCREMENT,
    estado_de_agendamiento VARCHAR(100)
);
```

- Tabla de servicios agendados

```sql
-- 10. Tabla de servicios agendados
CREATE TABLE servicios_agendados (
 id_agendamiento INT PRIMARY KEY AUTO_INCREMENT,
 id_mascota INT,
 id_servicio INT,
 fecha_hora DATETIME,
 id_estado_agendamiento INT NOT NULL,
 FOREIGN KEY (id_mascota) REFERENCES mascotas(id_mascota),
 FOREIGN KEY (id_servicio) REFERENCES servicios(id_servicio),
 FOREIGN KEY (id_estado_agendamiento) REFERENCES estado_agendamiento(id_estado_agendamiento)
);
```

- Tabla de medicamentos

```sql
-- 11. Tabla de medicamentos
CREATE TABLE medicamentos (
 id_medicamento INT PRIMARY KEY AUTO_INCREMENT,
 nombre VARCHAR(100) NOT NULL,
 descripcion TEXT,
 stock INT DEFAULT 0,
 precio_unitario FLOAT
);
```

- Tabla Medicamentos suministrados durante las citas

```sql
-- 12. Medicamentos suministrados durante citas
CREATE TABLE medicamentos_suministrados (
 id_registro INT PRIMARY KEY AUTO_INCREMENT,
 id_cita INT,
 id_medicamento INT,
 dosis VARCHAR(50),
 observaciones TEXT,
 FOREIGN KEY (id_cita) REFERENCES citas(id_cita),
 FOREIGN KEY (id_medicamento) REFERENCES medicamentos(id_medicamento)
);
```

- Tabla de Historial clínico

```sql
-- 13. Historial clínico
CREATE TABLE historial_clinico (
 id_historial INT PRIMARY KEY AUTO_INCREMENT,
 id_cita INT,
 fecha_registro DATE DEFAULT (CURRENT_DATE),
 descripcion TEXT,
 FOREIGN KEY (id_cita) REFERENCES citas(id_cita)
);
```

- Tabla de Tipo de pago

```sql
-- 14. TABLA DE ESTADO DE AGENDAMIENTO
CREATE TABLE tipo_pago (
    id_pago INT PRIMARY KEY AUTO_INCREMENT,
    tipo_de_pago VARCHAR(100)
);
```

- Tabla de Factura

```sql
-- 15. tabla de Facturación
CREATE TABLE factura (
 id_factura INT PRIMARY KEY AUTO_INCREMENT,
 id_propietario INT,
 id_mascota INT,
 id_cita INT,
 fecha_emision DATETIME DEFAULT CURRENT_TIMESTAMP,
 total DECIMAL(10,2),
 id_pago INT NOT NULL,
 id_agendamiento INT,
 FOREIGN KEY (id_propietario) REFERENCES propietarios(id_propietario),
 FOREIGN KEY (id_pago) REFERENCES tipo_pago(id_pago),
 FOREIGN KEY (id_mascota) REFERENCES mascotas(id_mascota),
 FOREIGN KEY (id_cita) REFERENCES citas(id_cita),
 FOREIGN KEY (id_agendamiento) REFERENCES servicios_agendados(id_agendamiento)
);
```

- Tabla detalle de la facturación

```sql
-- 16. Tabla de detalle de la facturacion
CREATE table detalle_facturación (
id_detalle INT PRIMARY KEY AUTO_INCREMENT,
id_factura INT,
id_servicio INT,
FOREIGN KEY (id_factura) REFERENCES factura(id_factura),
FOREIGN KEY (id_servicio) REFERENCES servicios(id_servicio)
);
```

4.2 Ingreso de información

4.2.1 Ingreso de la información por clausula "INSERT INTO"

Ahora, procederemos ingresar toda la información por partes a las tablas, en primera instancia haremos uso de la clausula "INSERT INTO" para rellenar todas las tablas independientes y que contengan registros específicos para interactuar con las otras tablas.

- Tabla de especialidad del veterinario

```sql
INSERT INTO especialidad_vet (nombre_especialidad)
VALUES
  ('cardiología'),
  ('gástrico'),
  ('cirugía'),
  ('vascular'),
  ('general');
```

- Tabla de estado de cita

```sql
INSERT INTO estado_cita (estado_de_cita) VALUES
  ('pendiente'),
  ('confirmada'),
  ('cancelada'),
  ('reprogramada'),
  ('completada'),
  ('no asistió');
```

- Tabla de motivo de cita

```sql
INSERT INTO motivo_cita (motivo_de_cita) VALUES
  ('vacunacion'),
  ('control general'),
  ('urgencia'),
  ('desparasitacion'),
  ('esterilizacion'),
  ('consulta de seguimiento'),
  ('cirugia menor'),
  ('revision postoperatoria');
```

- Tabla de tipo de pago

```sql
INSERT INTO tipo_pago (tipo_de_pago) VALUES
  ('efectivo'),
  ('tarjeta de credito'),
  ('tarjeta de debito'),
  ('transferencia bancaria'),
  ('pago movil'),
  ('nequi'),
  ('daviplata');
```

- Tabla de Servicios

```sql
  INSERT INTO servicios (nombre, descripcion, precio)
VALUES
  ('Consulta General', 'Revisión médica general para mascotas.', 80000.00),
  ('Consulta de Control', 'Evaluación médica periódica para monitorear la salud de la mascota.', 70000.00),
  ('Vacunación', 'Aplicación de vacunas esenciales para perros y gatos.', 50000.00),
  ('Desparasitación Interna', 'Eliminación de parásitos intestinales mediante tratamiento oral.', 40000.00),
  ('Desparasitación Externa', 'Aplicación de antiparasitarios tópicos contra pulgas y garrapatas.', 45000.00),
  ('Cirugía Menor', 'Procedimientos quirúrgicos menores con anestesia local o general.', 250000.00),
  ('Esterilización', 'Cirugía de castración/esterilización para perros o gatos.', 300000.00),
  ('Peluquería Canina', 'Baño, corte de pelo, uñas y limpieza general.', 60000.00),
  ('Urgencias 24h', 'Atención veterinaria inmediata fuera del horario habitual.', 100000.00),
  ('Control de Peso', 'Asesoramiento y seguimiento nutricional para mascotas con sobrepeso.', 70000.00),
  ('Limpieza Dental', 'Higiene bucal profesional para prevenir enfermedades dentales.', 120000.00),
  ('Consulta a Domicilio', 'Visita veterinaria en casa para comodidad del cliente.', 90000.00);
```

- Tabla de medicamentos

```sql
INSERT INTO medicamentos (nombre, descripcion, stock, precio_unitario)
VALUES
  ('Ivermectina', 'Antiparasitario de amplio espectro para perros y gatos.', 50, 18000.00),
  ('Enrofloxacina', 'Antibiótico de amplio espectro para infecciones bacterianas.', 30, 25000.00),
  ('Carprofeno', 'Antiinflamatorio no esteroideo para el manejo del dolor.', 40, 32000.00),
  ('Meloxicam', 'Analgésico y antiinflamatorio en suspensión oral.', 25, 28000.00),
  ('Fipronil', 'Tratamiento tópico contra pulgas y garrapatas.', 60, 22000.00),
  ('Clindamicina', 'Antibiótico para infecciones de piel y heridas.', 35, 24000.00),
  ('Metronidazol', 'Antibiótico y antiparasitario gastrointestinal.', 45, 16000.00),
  ('Ketoconazol', 'Antifúngico para infecciones dérmicas.', 20, 21000.00),
  ('Prednisolona', 'Corticoide para procesos inflamatorios y alérgicos.', 28, 19000.00),
  ('Amoxicilina + Clavulánico', 'Antibiótico potente de amplio espectro.', 38, 30000.00),
  ('Sucralfato', 'Protector gástrico para animales con úlceras o gastritis.', 22, 18000.00),
  ('Omeprazol', 'Inhibidor de la secreción ácida gástrica.', 26, 17000.00),
  ('Levamisol', 'Desparasitante oral de uso veterinario.', 33, 15000.00),
  ('Florfenicol', 'Antibiótico de amplio espectro inyectable.', 18, 35000.00),
  ('Doxiciclina', 'Antibiótico para infecciones respiratorias y urinarias.', 29, 27000.00),
  ('Ranitidina', 'Inhibidor de la secreción ácida gástrica.', 20, 16000.00),
  ('Acepromazina', 'Sedante ligero para procedimientos menores.', 14, 23000.00),
  ('Tramadol', 'Analgésico para manejo del dolor moderado a severo.', 17, 29000.00),
  ('Lactulosa', 'Laxante para tratar el estreñimiento en mascotas.', 19, 18000.00),
  ('Toltrazuril', 'Anticoccidial para tratamiento de parásitos intestinales.', 23, 26000.00);
```

- Tabla de estado de agendamiento

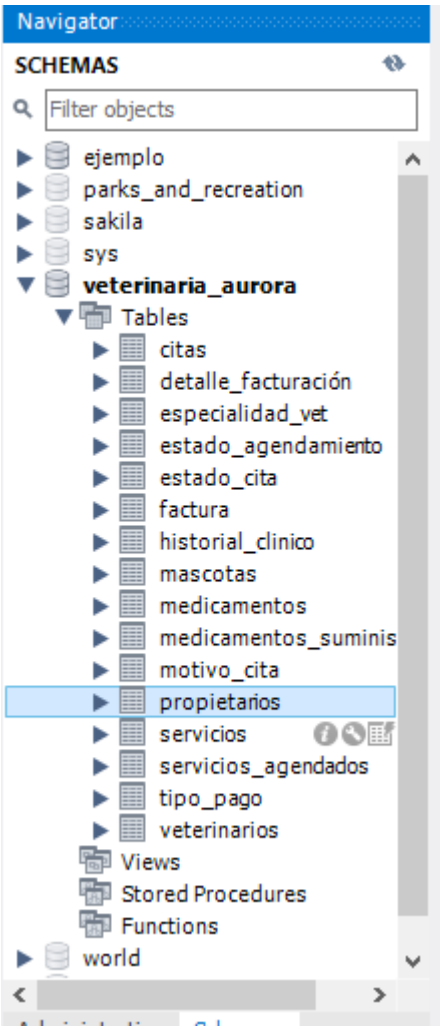
```sql
INSERT INTO estado_agendamiento (id_estado_agendamiento, estado_de_agendamiento)
VALUES
  (1, 'Pendiente'),
  (2, 'Confirmado'),
  (3, 'Cancelado'),
  (4, 'Reprogramado'),
  (5, 'No Asistió'),
  (6, 'Atendido'),
  (7, 'En Sala de Espera'),
  (8, 'En Proceso'),
  (9, 'Finalizado'),
  (10, 'Derivado a Especialista'),
  (11, 'Suspendido'),
  (12, 'Recordatorio Enviado');
```

4.2.1 Ingreso de la información por clausula "INSERT INTO"

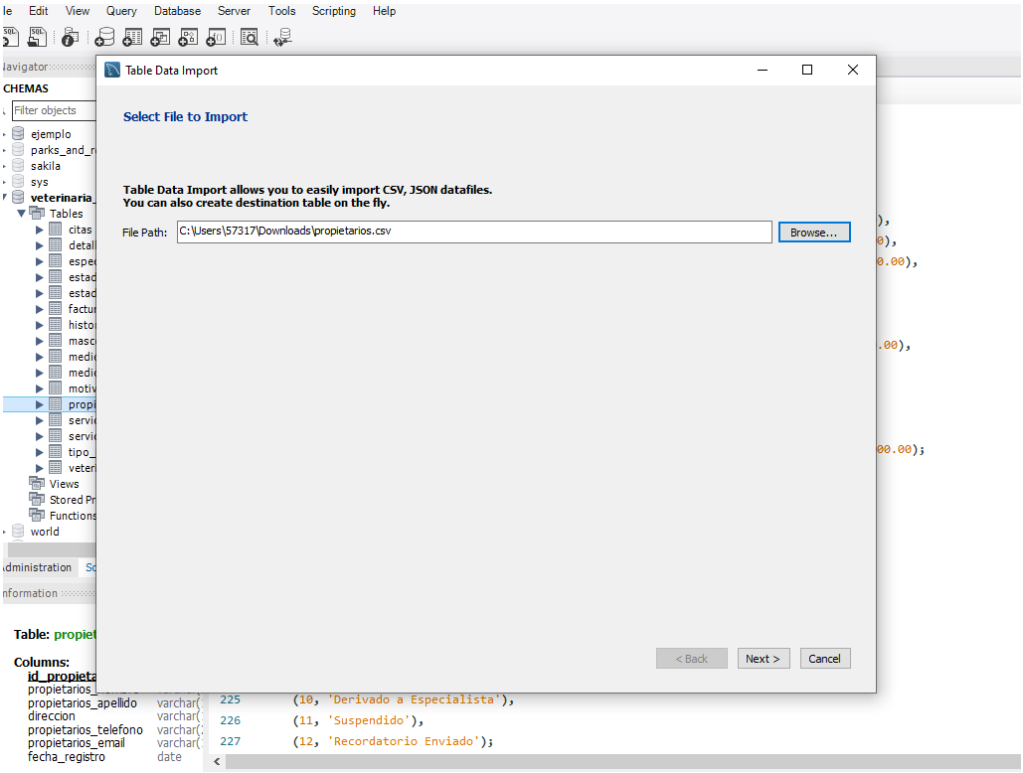
Ahora llenaremos el restante de las tablas con archivos CSV creados con la librería Faker en Python.

- En primer lugar, lo que haremos será ir a la pantalla donde se despliega las bases de datos dentro del programa, buscaremos la base de datos que estemos trabajando,

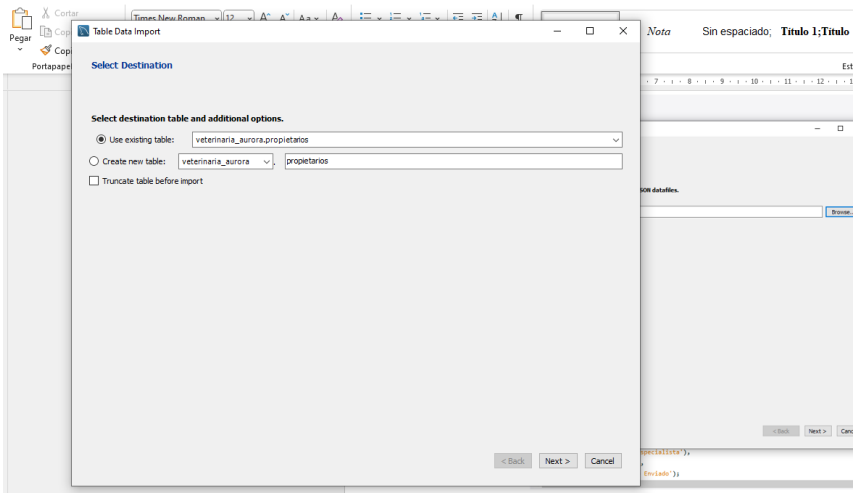
abrimos la pestaña “Tables” y nos paramos en la tabla que queramos rellenar, en este ejemplo será la tabla de propietarios.



Damos click derecho y se desplegará un menú de opciones donde seleccionaremos la opción “Table Data Import Wizard”, después se nos abrirá una nueva pantalla donde nos pedirá el path del archivo que contiene la información deseada, hacemos click en “Browse” y seleccionamos el archivo.

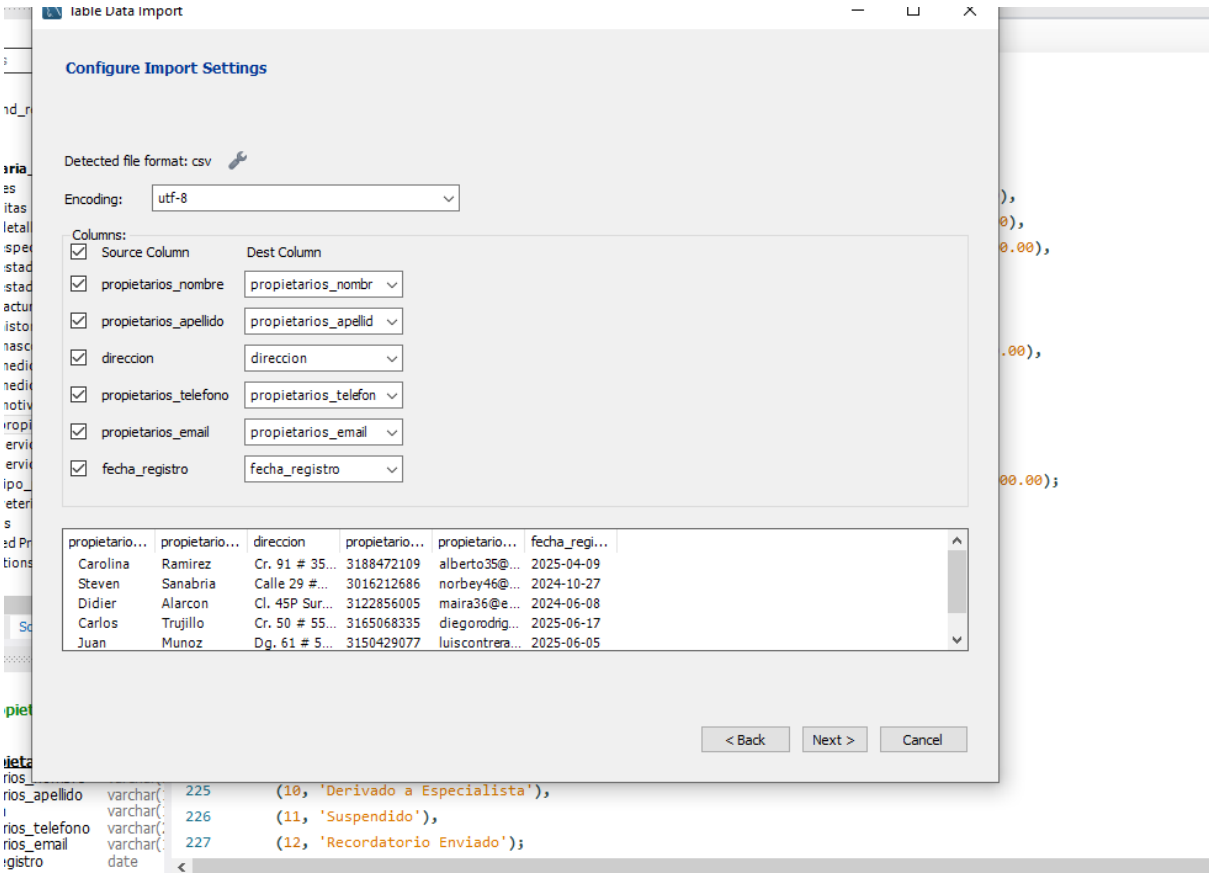


Damos click en “Next” y elegimos en la siguiente pantalla la opción “Use existing table”, seleccionamos la opción que corresponda a la tabla de propietarios previamente creada y damos click en “Next” nuevamente.



En la siguiente pantalla, haremos match de las columnas en el archivo CSV con las columnas de la tabla de SQL, teniendo cuidado ya que deben corresponder exactamente o si no los datos

no se importarán, aquí podremos revisar la información en las columnas, después damos click en “Next”.



Damos “Next” nuevamente, y finalmente se importan todos los datos. Este proceso se repetirá para cada tabla que falte por rellenar.

4.3 Consultas

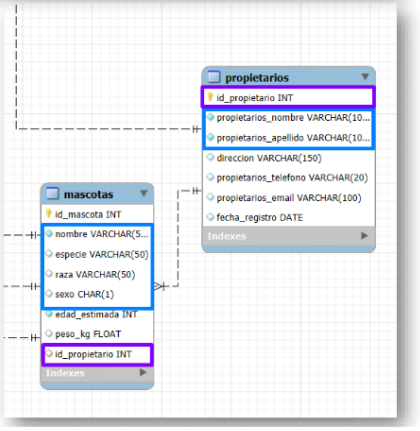
Para finalizar, veremos unas cuantas consultas bastante comunes dentro de las empresas a la hora de manipular un archivo SQL. Algunas son especificas para el negocio de la veterinaria, sin embargo, el concepto es el mismo. Si quisiéramos guardar alguna de las siguientes consultas para acceder rápidamente, solo haría falta usar la cláusula “CREATE VIEW”.

1 –Obtener todas las mascotas con el nombre de su propietario

JOIN

```
1   SELECT m.nombre AS nombre_mascota, m.sexo, m.especie, m.raza, pr.propietarios_nombre, pr.propietarios_apellido
2   FROM mascotas AS m  JOIN  propietarios AS pr
3   ON m.id_propietario = pr.id_propietario;
```

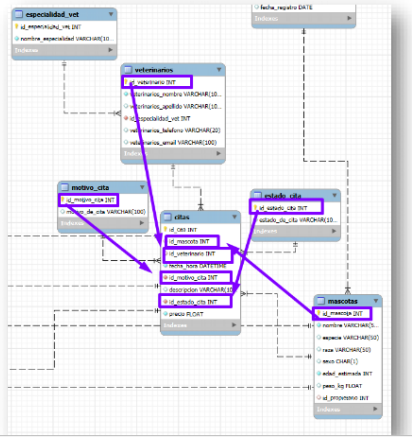
nombre_mascota	sexo	especie	raza	propietarios_nombre	propietarios_apellido
Melissa	F	Conejo	Rex	Camilo	Posada
Danny	F	Perro	Beagle	Oscar	Caballero
Alicia	M	Gato	Siamaes	Maria	Gil
Mary	M	Perro	Beagle	Jose	Arevalo
Lisa	M	Gato	Persa	Jimena	Padilla
Jose	F	Gato	Persa	Yolanda	Paez
Kimberly	F	Conejo	Angora	Karen	Camacho
Matthew	M	Perro	Labrador Retriever	Ivan	Camacho
Reginald	F	Perro	Labrador Retriever	Lucia	Rivas
Charles	F	Ave	Loro	Orlando	Erazo



2-Ver citas con nombre de mascota y veterinario

```
use veterinaria_aurora;
SELECT  c.id_cita, c.fecha_hora, m.nombre as nombre_de_mascota, vet.veterinarios_nombre, vet.veterinarios_apellido,
 est.estado_de_cita, mt.motivo_de_cita
FROM citas AS c
JOIN mascotas AS m ON c.id_mascota = m.id_mascota
JOIN veterinarios AS vet ON vet.id_veterinario = c.id_veterinario
JOIN estado_cita AS est ON c.id_estado_cita = est.id_estado_cita
JOIN motivo_cita AS mt ON c.id_motivo_cita = mt.id_motivo_cita;
```

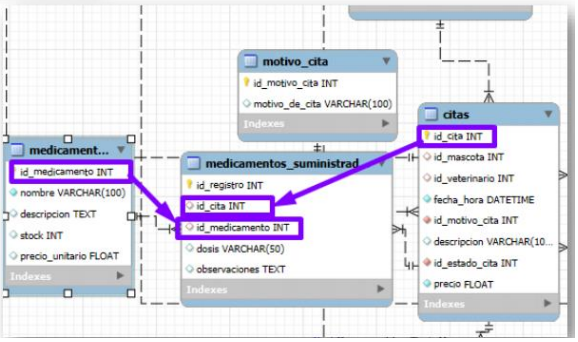
id_cita	fecha_hora	nombre_de_mascota	veterinarios_nombre	veterinarios_apellido	estado_de_cita	motivo_de_cita
1	2025-08-06 10:20:43	Stephen	Gracela	Romero	pendiente	esterilizacion
3	2024-12-03 13:25:56	Alicia	Eugenia	Morales	pendiente	consulta de seguimiento
5	2025-01-19 05:12:23	Lisa	Javier	Castro	pendiente	consulta de seguimiento
11	2025-07-17 02:25:48	David	Manuel	Alvarez	confirmada	cirugia menor
13	2025-08-09 21:49:02	Jose	Pedro	Delgado	confirmada	revision postoperatoria
2	2024-10-26 16:41:10	Danny	Luz	Diaz	cancelada	control general
7	2024-12-23 14:25:07	Matthew	Javier	Castro	cancelada	cirugia menor
14	2025-03-28 04:32:26	Jose	Luz	Diaz	cancelada	desparasitacion
17	2024-10-24 22:02:33	Matthew	Javier	Castro	cancelada	cirugia menor
18	2025-05-23 18:04:47	David	Rafael	Perez	cancelada	cirugia menor
4	2025-02-11 17:52:27	Eric	Javier	Jimenez	reprogramada	cirugia menor
6	2025-09-17 07:22:48	Matthew	Eugenia	Morales	reprogramada	revision postoperatoria
8	2024-11-04 07:55:43	Katherine	Maira	Pena	reprogramada	cirugia menor
12	2025-07-27 07:53:49	Mary	Catalina	Sanchez	reprogramada	urgencia
9	2025-03-16 19:15:57	Jason	Luz	Diaz	completada	cirugia menor
15	2024-11-23 17:57:57	Taylor	Angelica	Alarcon	completada	consulta de seguimiento
16	2025-08-26 16:47:03	Jose	Gracela	Romero	completada	urgencia
10	2025-08-05 21:07:22	Martin	Javier	Castro	no asistió	consulta de seguimiento
19	2025-01-28 08:23:51	David	Brayan	Londono	no asistió	control general



3- Medicamentos suministrados en cada cita

```
SELECT c.id_cita, c.fecha_hora, med.nombre AS medicamentos
FROM citas AS c
JOIN medicamentos_suministrados AS ms ON ms.id_cita = c.id_cita
JOIN medicamentos AS med ON ms.id_medicamento = med.id_medicamento
ORDER BY id_cita ASC;
```

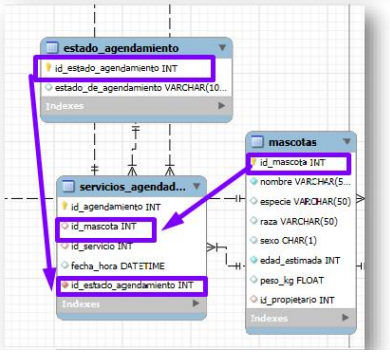
id_cita	fecha_hora	medicamentos
2	2024-10-26 16:41:10	Meloxicam
2	2024-10-26 16:41:10	Clindamicina
4	2025-02-11 17:52:27	Metronidazol
5	2025-01-19 05:12:23	Ivermectina
6	2025-09-17 07:22:48	Sucralfato
6	2025-09-17 07:22:48	Carprofeno
7	2024-12-23 14:25:07	Ivermectina
9	2025-03-16 19:15:57	Levamisol



4-Servicios agendados por cada mascota

```
SELECT sag.fecha_hora, ser.nombre AS servicios_agendados , estag.estado_de_agendamiento , m.nombre AS Mascota
FROM mascotas AS m
JOIN servicios_agendados AS sag ON sag.id_mascota = m.id_mascota
JOIN servicios AS ser ON ser.id_servicio = sag.id_servicio
JOIN estado_agendamiento AS estag ON estag.id_estado_agendamiento = sag.id_estado_agendamiento;
```

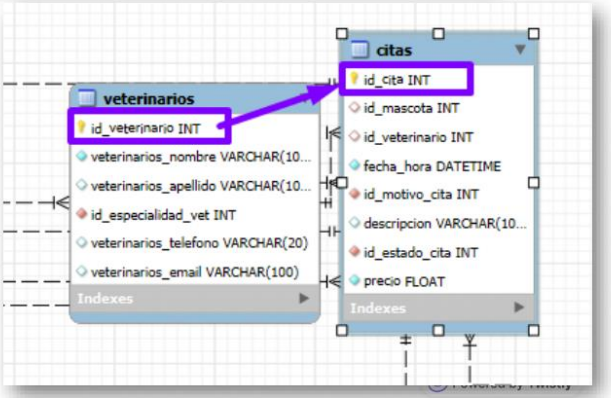
fecha_hora	servicios_agendados	estado_de_agendamiento	Mascota
2025-10-16 22:42:13	Consulta General	En Sala de Espera	Eric
2025-09-21 20:47:02	Consulta de Control	Pendiente	Matthew
2025-10-14 09:29:29	Consulta de Control	Derivado a Especialista	Reginald
2025-09-25 10:30:04	Vacunación	En Proceso	Danny
2025-09-22 22:14:05	Vacunación	Derivado a Especialista	Reginald
2025-11-05 16:38:59	Desparasitación Externa	No Asistió	Eric
2025-10-14 14:57:26	Desparasitación Externa	Recordatorio Enviado	David
2025-10-06 08:19:11	Desparasitación Externa	Pendiente	Katherine
2025-09-25 00:19:38	Desparasitación Externa	Suspendido	Kimberly
2025-09-29 07:16:33	Desparasitación Externa	Confirmado	Jerry
2025-10-17 02:04:10	Desparasitación Externa	Reprogramado	Stephen
2025-10-13 00:29:11	Desparasitación Externa	Cancelado	Lisa
2025-09-20 16:46:51	Desparasitación Externa	Reprogramado	Stephen



5- Veterinarios que han atendido mascotas

```sql
SELECT veterinarios_nombre, veterinarios_apellido
FROM veterinarios
WHERE id_veterinario IN (SELECT id_veterinario FROM citas);
```

veterinarios_nombre	veterinarios_apellido
Angelica	Alarcon
Brayan	Londono
Eugenia	Morales
Graciela	Romero
Luz	Diaz
Manuel	Alvarez
Catalina	Sanchez
Pedro	Delgado
Javier	Castro
Maira	Pena
Rafael	Perez
Javier	Jimenez

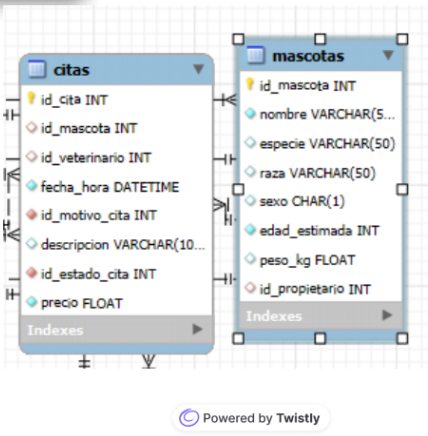


6- Mascotas que tienen más de 2 citas

```sql
-- 6- Mascotas que tienen más de 2 citas
SELECT nombre, especie
FROM mascotas
WHERE id_mascota IN (SELECT id_mascota FROM citas GROUP BY id_mascota HAVING COUNT(id_mascota) > 2
);
```

id_mascota	nombre	especie	raza	sexo	edad_estimada	peso_kg	id_propietario
6	Jose	Gato	Persa	F	14	44.2	6
11	David	Ave	Canario	M	10	45.7	11
14	Matthew	Ave	Loro	M	14	22.8	14
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

id_cita	id_mascota	id_veterinario	fecha_hora	id_motivo_cita	descripcion	id_estado_cita	precio
2	2	10	2024-10-26 16:41:10	2	Occaecati consequatur ipsam placeat. Dicta quo...	3	35033.1
3	3	5	2024-12-03 13:25:56	6	Architecto similique laudantium labore quas pari...	1	275425
12	4	13	2025-07-27 07:53:49	3	Sequi perspiciatis id error natus fugit. Ad quae ...	4	227180
5	5	15	2025-01-19 05:12:23	6	Alias dolore eum. Magni quisquam facilis nobis. ...	1	89022.1
13	6	14	2025-08-09 21:49:02	8	Laborum rerum excepturi eos. Repudiandae offi...	2	206137
14	6	10	2025-03-28 04:32:26	4	Pariatur ducimus assumenda aliquam provident ...	3	275239
16	6	9	2025-08-26 16:47:03	3	Itaque quos accusantium quo necessitatibus sint.	5	33843.9
20	9	11	2025-03-29 10:07:43	5	Nemo natus reiciendis nulla. Dicta voluptate eos...	6	228805
11	11	11	2025-07-17 02:25:48	7	Similique suscipit corporis excepturi nisi tempore.	2	264713
18	11	17	2025-05-23 18:04:47	7	Sequi excepturi illo ea. Neque excepturi illo. Ma...	3	74414
19	11	3	2025-01-28 08:23:51	2	Aspernatur eveniet nemo beatae. Ipsam quos r...	6	184866
4	12	18	2025-02-11 17:52:27	7	Blanditiis non ducimus libero autem quia repellat.	4	140435
1	13	9	2025-08-06 10:20:43	5	Impedit sit perferendis reiciendis.	1	286014
6	14	5	2025-09-17 07:22:48	8	Nostrum molestiae veritatis natus. Error explica...	4	187882
7	14	15	2024-12-23 14:25:07	7	Nemo animi aperiam eaque ab. Eius nemo aliqu...	3	275181
17	14	15	2024-10-24 22:02:33	7	Dolorum facere sequi praesentium eaque. Nihil ...	3	108036
15	15	2	2024-11-23 17:57:57	6	Molestiae accusamus eligendi dolore nostrum eni...	5	280892
9	16	10	2025-03-16 19:15:57	7	Hic non voluptates voluptatibus recusandae offi...	5	273197
10	18	15	2025-08-05 21:07:22	6	Occaecati incidunt iure corrupti eveniet. Quo co...	6	299779
8	19	16	2024-11-04 07:55:43	7	Architecto consequatur distinctio quisquam tem...	4	266855



7- Medicamentos sin stock

```sql
-- 7- Medicamentos sin stock
SELECT *
FROM medicamentos WHERE stock = 0;
```

id_medicamento	nombre	descripcion	stock	precio_unitario
NULL	NULL	NULL	NULL	NULL

id_medicamento	nombre	descripcion	stock	precio_unitario
1	Ivermectina	Antiparasitario de amplio espectro para perros ...	50	18000
2	Enrofloxacina	Antibiótico de amplio espectro para infecciones ...	30	25000
3	Carprofeno	Antiinflamatorio no esteroideo para el manejo d...	40	32000
4	Meloxicam	Analgésico y antiinflamatorio en suspensión oral.	25	28000
5	Fipronil	Tratamiento tópico contra pulgas y garrapatas.	60	22000
6	Clindamicina	Antibiótico para infecciones de piel y heridas.	35	24000
7	Metronidazol	Antibiótico y antiparasitario gastrointestinal.	45	16000
8	Ketoconazol	Antifúngico para infecciones dérmicas.	20	21000
9	Prednisolona	Corticoide para procesos inflamatorios y alérgicos.	28	19000
10	Amoxicilina + Clavulánico	Antibiótico potente de amplio espectro.	38	30000
11	Sucralfato	Protector gástrico para animales con úlceras o g...	22	18000
12	Omeprazol	Inhibidor de la secreción ácida gástrica.	26	17000
13	Levamisol	Desparasitante oral de uso veterinario.	33	15000
14	Florfenicol	Antibiótico de amplio espectro inyectable.	18	35000
15	Doxiciclina	Antibiótico para infecciones respiratorias y urina...	29	27000
16	Ranitidina	Inhibidor de la secreción ácida gástrica.	20	16000
17	Acepromazina	Sedante ligero para procedimientos menores.	14	23000
18	Tramadol	Analgésico para manejo del dolor moderado a s...	17	29000
19	Lactulosa	Laxante para tratar el estreñimiento en mascotas.	19	18000
20	Toltrazuril	Anticoccidial para tratamiento de parásitos intes...	23	26000
NULL	NULL	NULL	NULL	NULL

Powered by Twistly

8- Facturas mayores al promedio

AVG(total)
143747.272500

```sql
-- Facturas mayores al promedio
SELECT AVG(total) FROM factura;
SELECT * FROM factura  WHERE total > (SELECT AVG(total) FROM factura);
```

id_factura	id_propietario	id_mascota	id_cita	fecha_emision	total	id_pago	id_agendamiento
2	2	2	2	2025-09-11 00:00:00	172457.04	3	2
3	3	3	3	2025-09-04 00:00:00	189110.41	5	3
4	4	4	4	2025-08-22 00:00:00	184707.86	2	4
6	6	6	6	2025-09-04 00:00:00	177833.70	1	6
9	9	9	9	2025-08-10 00:00:00	173057.31	2	9
10	10	10	10	2025-08-25 00:00:00	187708.70	3	10
14	14	14	14	2025-08-17 00:00:00	182763.95	2	14
16	16	16	16	2025-09-09 00:00:00	162634.93	4	16
18	18	18	18	2025-08-04 00:00:00	198610.23	2	18
19	19	19	19	2025-09-03 00:00:00	149761.26	1	19
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Powered by Twistly

9- Total acumulado de facturación por propietario

```sql
SELECT
    p.id_propietario,
    CONCAT(p.propietarios_nombre, ' ', p.propietarios_apellido) AS propietario,
    (
        SELECT SUM(f.total)
        FROM factura f
        WHERE f.id_propietario = p.id_propietario
    ) AS total_acumulado
FROM propietarios p;
```

id_propietario	propietario	total_acumulado
1	Camilo Posada	131683.49
2	Oscar Caballero	172457.04
3	Maria Gil	189110.41
4	Jose Arevalo	184707.86
5	Jimena Padilla	96132.99
6	Yolanda Paez	177833.70
7	Karen Camacho	103262.87
8	Ivan Camacho	128424.52
9	Lucia Rivas	173057.31
10	Orlando Erazo	187708.70
11	Margarita Molina	104370.42
12	Roger Ramirez	128605.79
13	Edwin Sanchez	130664.31
14	Alexis Torres	182763.95
15	Cecilia Marin	112528.81
16	Isaac Gonzalez	162634.93
17	Roberto Sando...	89500.70
18	Carlos Corredor	198610.23

Powered by Twistly

10- Ranking de citas por mascota

```sql
SELECT m.id_mascota, m.nombre AS Paciente, m.especie, m.raza, COUNT(c.id_cita) AS numero_de_citas
FROM citas c
JOIN mascotas m ON c.id_mascota = m.id_mascota
GROUP BY m.id_mascota, m.nombre, m.especie, m.raza
ORDER BY numero_de_citas DESC;
```

id_mascota	Paciente	especie	raza	numero_de_citas
6	Jose	Gato	Persa	3
11	David	Ave	Canario	3
14	Matthew	Ave	Loro	3
2	Danny	Perro	Beagle	1
3	Alicia	Gato	Siamaes	1
4	Mary	Perro	Beagle	1
5	Lisa	Gato	Persa	1
9	Reginald	Perro	Labrador Retriever	1
12	Eric	Perro	Labrador Retriever	1
13	Stephen	Perro	Beagle	1
15	Taylor	Hamster	Ruso	1
16	Jason	Ave	Canario	1
18	Martin	Ave	Periquito	1
19	Katherine	Ave	Periquito	1

Powered by Twistly

11- Crear vista de historial clínico extendido

```
SELECT h.id_historial, h.fecha_registro AS Fecha_ingreso_al_sistema, h.descripcion AS observaciones,
c.id_cita, c.fecha_hora AS hora_de_la_cita, c.descripcion,
mo.motivo_de_cita,
est.estado_de_cita,
m.nombre AS paciente, m.especie, m.raza, m.sexo, m.edad_estimada AS edad, m.peso_kg,
v.veterinarios_nombre AS nombre_del_veterinario, v.veterinarios_apellido AS apellido_del_veterinario,
meds.id_medicamento, med.nombre AS medicamento_suministrado, med.descripcion AS especificaciones, meds.dosis
FROM citas c
JOIN motivo_cita mo ON c.id_motivo_cita = mo.id_motivo_cita
JOIN mascotas m ON c.id_mascota = m.id_mascota
JOIN veterinarios v ON c.id_veterinario
JOIN estado_cita est ON c.id_estado_cita
JOIN historial_clinico h ON c.id_cita =
JOIN medicamentos_suministrados meds ON
JOIN medicamentos med ON meds.id_medicam
ORDER BY h.id_historial ASC;
```

id_historial	Fecha_ingreso_al_sistema	observaciones	id_cita	hora_de_la_cita	descripcion
1	2024-02-10	Vamos tomar hacerlo ambiente crecimiento emp...	4	2025-02-11 17:52:27	Blanditiis non ducimus libero autem q
2	2024-04-27	Libertad origen ley juan solo todo.	10	2025-08-05 21:07:22	Occaecati incidunt iure corrupti ever
3	2024-05-19	Todas cuenta octubre rodrÃ-guez unos eso dejÃ³.	9	2025-03-16 19:15:57	Hic non voluptates voluptatibus recu
4	2024-09-11	SerÃ-a arte les humana cualquier proceso euro...	7	2024-12-23 14:25:07	Nemo animi aperiam eaque ab. Eius
6	2025-02-02	AtrÃis particular podÃ-a nivel media empresa n...	6	2025-09-17 07:22:48	Nostrum molestiae veritatis natus. E
6	2025-02-02	AtrÃis particular podÃ-a nivel media empresa n...	6	2025-09-17 07:22:48	Nostrum molestiae veritatis natus. E
7	2025-05-28	Francia informe manuel paÃ-s julio estilo casi to...	4	2025-02-11 17:52:27	Blanditiis non ducimus libero autem q
8	2025-01-25	Nuevos dice acuerdo salud llamado este lo hubo.	12	2025-07-27 07:53:49	Sequi perspiciatis id error natus fugi
9	2023-10-22	Intereses experiencia servicios incluso lÃ-nea.	18	2025-05-23 18:04:47	Sequi excepturi illo ea. Neque excep
9	2023-10-22	Intereses experiencia servicios incluso lÃ-nea.	18	2025-05-23 18:04:47	Sequi excepturi illo ea. Neque excep
10	2024-06-24	N mayor buscar precio mis carta dÃ-a mÃis.	18	2025-05-23 18:04:47	Sequi excepturi illo ea. Neque excep
10	2024-06-24	N mayor buscar precio mis carta dÃ-a mÃis.	18	2025-05-23 18:04:47	Sequi excepturi illo ea. Neque excep
14	2024-06-20	Cierta puso investigaciÃ³n sistemas militares.	7	2024-12-23 14:25:07	Nemo animi aperiam eaque ab. Eius
15	2025-06-20	Nuevo tambiÃ©n problema lÃ-der travÃ©s ver...	18	2025-05-23 18:04:47	Sequi excepturi illo ea. Neque excep
15	2025-06-20	Nuevo tambiÃ©n problema lÃ-der travÃ©s ver...	18	2025-05-23 18:04:47	Sequi excepturi illo ea. Neque excep
16	2025-05-29	Hicieron siquiera tendrÃi sol verdad resulta.	18	2025-05-23 18:04:47	Sequi excepturi illo ea. Neque excep
16	2025-05-29	Hicieron siquiera tendrÃi sol verdad resulta.	18	2025-05-23 18:04:47	Sequi excepturi illo ea. Neque excep

Powered by Twistly

12- Vista de servicios realizados por mascota

```
SELECT agen.id_agendamiento, agen.fecha_hora,
m.nombre AS nombre_mascota, m.especie, m.raza,
serv.nombre AS servicio_realizado, serv.descripcion, serv.precio,
est.estado_de_agendamiento
FROM servicios_agendados agen
JOIN mascotas m ON agen.id_mascota = m.id_mascota
JOIN servicios serv ON agen.id_servicio = serv.id_servicio
JOIN estado_agendamiento est ON agen.id_estado_agendamiento = est.id_estado_agendamiento
ORDER BY id_agendamiento ASC;
```

id_agendamiento	fecha_hora	nombre_mascota	especie	raza	servicio_realizado	descripcion
1	2025-09-21 20:47:02	Matthew	Ave	Loro	Consulta de Control	Evaluación médica periódica para monit
2	2025-11-11 19:33:09	Taylor	Hamster	Ruso	Consulta a Domicilio	Visita veterinaria en casa para comodic
3	2025-11-05 16:38:59	Eric	Perro	Labrador Retriever	Desparasitación Externa	Aplicación de antiparasitarios tópicos o
4	2025-10-14 09:29:29	Reginald	Perro	Labrador Retriever	Consulta de Control	Evaluación médica periódica para monit
5	2025-10-14 14:57:26	David	Ave	Canario	Desparasitación Externa	Aplicación de antiparasitarios tópicos o
6	2025-10-06 08:19:11	Katherine	Ave	Periquito	Desparasitación Externa	Aplicación de antiparasitarios tópicos o
7	2025-09-25 00:19:38	Kimberly	Conejo	Angora	Desparasitación Externa	Aplicación de antiparasitarios tópicos o
8	2025-09-29 07:16:33	Jerry	Ave	Loro	Desparasitación Externa	Aplicación de antiparasitarios tópicos o
9	2025-10-17 02:04:10	Stephen	Perro	Beagle	Desparasitación Externa	Aplicación de antiparasitarios tópicos o
10	2025-09-25 10:30:04	Danny	Perro	Beagle	Vacunación	Aplicación de vacunas esenciales para
11	2025-10-16 22:42:13	Eric	Perro	Labrador Retriever	Consulta General	Revisión médica general para mascotas
12	2025-09-22 22:14:05	Reginald	Perro	Labrador Retriever	Vacunación	Aplicación de vacunas esenciales para
13	2025-10-26 06:32:40	David	Ave	Canario	Consulta a Domicilio	Visita veterinaria en casa para comodic
14	2025-11-09 18:05:49	Jason	Ave	Canario	Cirugía Menor	Procedimientos quirúrgicos menores co
15	2025-10-13 00:29:11	Lisa	Gato	Persa	Desparasitación Externa	Aplicación de antiparasitarios tópicos o
16	2025-11-06 19:18:03	Amber	Hamster	Ruso	Control de Peso	Asesoramiento y seguimiento nutricion
17	2025-09-28 22:01:46	Danny	Perro	Beagle	Peluquería Canina	Baño, corte de pelo, uñas y limpieza ge

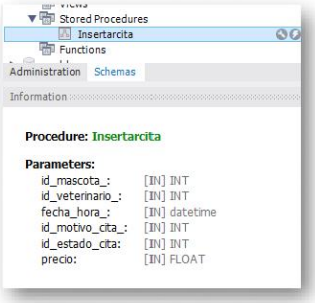
Powered by Twistly

13- Crear procedimiento almacenado para registrar nueva cita

```
CREATE PROCEDURE registrar_cita (
    IN p_id_mascota INT,
    IN p_id_veterinario INT,
    IN p_fecha_hora DATETIME,
    IN p_id_motivo_cita INT,
    IN p_descripcion VARCHAR(100),
    IN p_id_estado_cita INT,
    IN p_precio FLOAT
)
BEGIN
    INSERT INTO citas (
        id_mascota,
        id_veterinario,
        fecha_hora,
        id_motivo_cita,
        descripcion,
        id_estado_cita,
        precio
```

```
    VALUES (
        p_id_mascota,
        p_id_veterinario,
        p_fecha_hora,
        p_id_motivo_cita,
        p_descripcion,
        p_id_estado_cita,
        p_precio
    );
END$$

DELIMITER ;
```




```
CALL Insertarcita(1, 5, '2025-08-06 10:20:43', 6, 4, 3400000);
```

Powered by Twistly

14- Trigger para reducir stock de medicamento al suministrarlo

```
DELIMITER //

CREATE TRIGGER reducir_stock
AFTER INSERT ON medicamentos_suministrados
FOR EACH ROW
BEGIN
    UPDATE medicamentos
    SET stock = stock - 1
    WHERE id_medicamento = NEW.id_medicamento;
END;
//

DELIMITER ;
```

Powered by Twistly

15- Trigger para actualizar factura automáticamente

```
DELIMITER $$

CREATE TRIGGER actualizar_total_factura
AFTER INSERT ON detalle_facturación
FOR EACH ROW
BEGIN
    DECLARE precio_servicio DECIMAL(10,2);

    -- Obtener el precio del servicio agregado
    SELECT s.precio INTO precio_servicio
    FROM servicios s
    WHERE s.id_servicio = NEW.id_servicio;

    -- Actualizar el total de la factura
    UPDATE factura
    SET total = IFNULL(total, 0) + precio_servicio
    WHERE id_factura = NEW.id_factura;
END$$

DELIMITER ;
```

```
INSERT INTO factura (id_propietario, id_mascota, id_cita, id_pago, total)
VALUES (1, 1, 1, 1, 0);

INSERT INTO detalle_facturación (id_factura, id_servicio)
VALUES (1, 3);  -- aquí 3 sería el id de un servicio (ej: 'Vacunación')
```

```
#Ejemplo
INSERT INTO factura (id_propietario, id_mascota, id_cita, id_pago, total)
VALUES (1, 1, 1, 1, 0);

SELECT id_factura, total
FROM factura
WHERE id_factura = 1;

INSERT INTO detalle_facturación (id_factura, id_servicio)
VALUES (1, 3);   -- el servicio con id=3 es 'Vacunación' (precio 50000.00)

SELECT id_factura, total
FROM factura
WHERE id_factura = 1;
```

id_factura	total
1	231683.49

id_factura	total
1	281683.49
NULL	NULL

Powered by Twistly